```
// index.php
$twig = new \Twig\Environment($loader, [
    'autoescape' => false,
]);
```

Now if you request the page, you'll see that the template now renders unsafely:

```
$ curl http://localhost:8080/index.php
...
  <p>Occupation: <script>window.alert("I control the browser now.")
  </script></p>
...
```

If Twig is running in this mode, data can be manually escaped by appending `|escape` or the short-form `|e` to the variable:

```
  <p>Occupation: {{ postData.occupation|e }}</p>
```

Conversely, you can skip autoescaping when it is enabled by appending `|raw` but your reasons had better be good. Twig offers several escape contexts, and these can be supplied as arguments when escaping with `|e()`. Use the modified `Twig\Environment` boilerplate (turning off autoescaping) as before with the following code:

```
// index.php
print $twig->render('escape.css.twig', [
    'selector' => 'body > main .link',
]);

print PHP_EOL;

print $twig->render('html-attr-escape.html.twig', [
    'srcAttr' => 'x" onerror"=alert(\'script injection\')"'
]);
```

Here we're rendering two different templates, `escape.css.twig`...

```
{# views/escape.css.twig #}
{{ selector|e('css') }} {
    font-weight: bold;
}
```